

DEPARTMENT OF COMPUTER SCIENCE FALL 2025

**CS4110 - High Performance
Computing with GPUs**

**Deliverable 2
Report**

By:

Fatima Sohail [23i-0633]

Hareem Zia [23f-0842]

HPC-A

The objective of this report is to document the comparative performance analysis between the CPU and a naive GPU implementation of the KLT algorithm. Two datasets of varying sizes were used to evaluate the performance before and after the naive implementation. The results provide an insight into scenarios when GPU acceleration is most beneficial and highlight why further optimisation techniques are necessary to maximise the speed up achievable on GPUs.

Our naive GPU implementation involves launching CUDA kernels for the functions `convolveImageHoriz`, `convolveImageVert`, and `trackFeatures`. Supporting functions required by these kernels were ported to the GPU to ensure parallel execution.

Execution Time Comparison

The average total execution times for both CPU-only and naive GPU implementations are summarised below.

Dataset	Features	Frames	CPU Average Total Time (s)	GPU Average Total Time (s)	Speed Up
images_provided	150	10	0.20	0.38	0.526x
images_traffic	100	550	34.98	13.51	2.589x

For the large dataset containing 550 frames, the GPU implementation achieved a 2.589x speedup, decreasing from 34.98 seconds on the CPU to 13.51 seconds on the GPU. This improvement highlights the advantage of parallel computation on GPUs, as the high volume of frames and features provide sufficient workload to utilise the GPU's computational cores. The 3 kernels benefit from concurrent execution across multiple threads, reducing overall processing latency.

On the contrary, the smaller dataset with only 10 frames showed a slower performance on the GPU, with total time increasing from 0.20 seconds (CPU) to 0.38 seconds (GPU). This suggests that the GPU kernel launch overhead and memory transfer overheads dominate when the workload is small, as the computation performed in each kernel is not large enough to offset the cost of transferring data between host and device memory, leading to underutilisation of resources.

Profiling Analysis

When comparing the two datasets, we saw clear differences in how the GPU handles small versus large workloads.

Dataset	Features	Frames	Average Total API call time (ms)	Average Total Kernel Launch time (ms)	Average Total Transfer Time (ms)
images_provided	150	10	0.2893	0.4016	0.0311
images_traffic	100	550	0.2063	0.1417	0.1176

For the 10-frame dataset (150 features), the API call time (289,391.2 ns) and kernel launch time (401,627.3 ns) are relatively high compared to the amount of actual data being processed. This indicates that most of the time is spent on overhead tasks such as memory setup and launching kernels, rather than on actual computation. This is because of frequent kernel launches over a small number of frames and each kernel processes very little workload. The data transfer time (31,114.1 ns) was small, showing that with smaller data, transferring information between the CPU and GPU does not have a major overhead. Overall, this dataset showed high overhead and low GPU utilization because the dataset is too small to benefit fully from parallel processing.

For the 550-frame dataset (100 features), the average API call time (206,356.5 ns) and kernel launch time (141,724.2 ns) both decrease significantly. This is due to less overhead per frame, since it is spread across a larger workload. For this dataset, kernel launches were not as frequent because there is a larger workload per kernel. This shows that the GPU performs more efficiently when it has a larger workload. However, the transfer time (117,641.3 ns) increases due to the larger amount of data being moved between the CPU (host) and GPU (device). Despite that, the GPU's overall performance improves because it is better utilized and spends more time on actual computation rather than setup and launches.

In short, smaller datasets have a higher overhead and underutilization of the GPU, while larger datasets achieve better efficiency at the cost of higher data transfer time. This means that for smaller datasets CPU-GPU communication overhead and kernel launches are the limiting factors and for larger datasets, transfer time is the limiting factor. This shows that GPU performance scales better with larger workloads, where the computational benefits outweigh the communication overhead.

Theoretical vs Measured Performance

The theoretical peak performance of the RTX 3080 is ~29.8 TFLOPS for single-precision operations.

For the images_transport dataset, analytically there are total 8.25×10^8 FLOPs. This results in an achieved performance of ~2.17GFLOPs resulting in only 0.0073% utilisation.

This clearly suggests that the gap between theoretical and measured performance is significant which needs further optimisation.

Conclusion

The GPU implementation achieved a decent speedup for the larger dataset, though the measured FLOPs were still significantly below the theoretical peak performance. This gap suggests that the naive implementation does not fully utilise the available computational resources, mainly due to memory transfer overheads and minute kernel optimisation. To decrease this gap, communication and memory optimization techniques should be used to exploit GPU resources.